

# 08 主流框架产 品化评测： LangGraph / AutoGen / AgentScope

## 评测的前提：别 被"谁的Star 多"带 偏

技术选型最常犯的错，是拿开源社区的Star 数或者论文引用量当决策依据。但产品经理该关心问题是：**这个框架能不能在我的团队里长期维护、出问题时能不能快速定位、迁移成本有多高。**这一章用产品化视角（不是学术视角）重新评测三个主流框架。

## 三个框架的核心设计哲学

|      | LangGraph                     | AutoG                         |
|------|-------------------------------|-------------------------------|
| 出身   | LangChain生态下的"图编排"扩展          | 微软出品期专注多体对话                   |
| 核心抽象 | 把Agent 的执行流程建模成 <b>状态图（节点</b> | 把 Agent ；成 <b>可对话色（Conver</b> |

|      | LangGraph                       | AutoG                           |
|------|---------------------------------|---------------------------------|
|      | <b>+边)</b> ，显式控制流转逻辑            | <b>Agent)</b> ，角色间传递驱动流         |
| 最大优势 | 流程可视化、状态管理清晰，适合"流程复杂但需要精确控制"的场景 | 多智能体的心智模型单直观（是"角色对话"），能快速搭建Demo |
| 最大局限 | 学习曲线较高，图状结构一开始设计不好后期改起来成本高      | 早期版本杂控制流如精确的分支）表偏弱，调角色对话时"发散"   |
| 语言生态 | Python (LangChain生态)            | Python                          |

## 产品化视角下的三个关键评测维度

### 1. 调试成本：出问题的时候，你能多快找到原因？

- **LangGraph**：状态图结构天然适合可视化调试——你能清楚看到"卡在哪个节点"。但如果图设计得过于复杂（节点/边太多），排查具体某次执行的状态转移仍然费时间。
- **AutoGen**：多角色对话模式下，调试的难点在于"到底是

哪个角色在哪句话上做了错误判断", 需要通读完整对话历史, 调试体验不如显式状态图直观。

- **AgentScope**: 提供了更完整的日志与监控体系 (这是它作为"工程导向框架"的优势), 排查生产问题相对更系统化。

## 2. 迁移成本: 如果框架不再维护了, 你的代码能救回来多少?

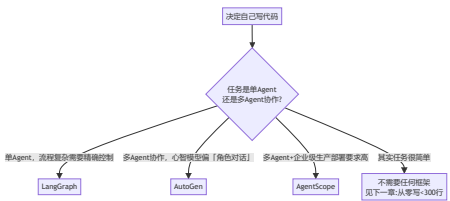
这是产品经理最容易忽略、但长期最致命的一个维度。三个框架都是"重度框架绑定"——你的业务逻辑往往会和框架的核心抽象 (状态图/对话角色/消息) 深度耦合。给自己留后路的做法是: **把真正的业务逻辑 (工具函数、Prompt 模板、数据处理) 写成独立于框架的纯函数, 框架只负责"编排调用顺序"这一层。** 这样即便未来换框架, 业务逻辑代码可以直接复用, 只需要重写"编排"那一层胶水代码。

## 3. 团队匹配度: 你的团队的技术背景, 跟框架的抽象方式合不合拍?

- 如果团队里有做过传统工作流引擎/状态机开发的人, **LangGraph 的图抽象会更容易上手。**

- 如果团队更习惯"角色扮演"式思考多智能体协作（比如做过游戏 NPC 对话系统），  
**AutoGen 的对话式抽象会更直觉。**
- 如果团队本身就需要企业级部署（分布式、高并发容错），  
**AgentScope 的工程化特性性价比更高。**

## 一个我自己会用的选型清单



## 一个反常识的建议：很多时候，你根本不需要框架

这是这一章最想传达的观点：**框架的价值是帮你处理"多Agent 协作的通信机制"、"复杂状态管理"这类工程复杂度，如果你的场景根本没有这些复杂度，引入框架只会增加不必要的抽象层和调试难度。** 我在自己的项目里，很多单Agent 场景最后选择了"不用任何框架，自己写一个几百行的最小实现"（就是下一章要讲的内容），原因很简单：**代码量小到可以完全掌控，出问题5分钟就能**

定位，不需要先理解一个框架的抽象模型才能排查问题。

## 产品经理清单

- ☐ 我引入框架的理由，是"这个框架解决了我场景里真实存在的复杂度"，还是"业界都在用，跟着用比较保险"？
- ☐ 如果框架未来废弃或团队想换框架，我的业务逻辑代码有多少是和框架深度耦合、拿不出来复用的？
- ☐ 我的团队现有技术背景，跟我选的框架的核心抽象方式，是不是天然匹配？如果不匹配，学习成本要算进选型决策里。

想看真实调用框架 SDK 跑起来的代码，而不只是文字对比表吗？[Demo08：用 LangGraph 真实实现「PRD 自动生成与多角色评审」](#) 用真实的 `langgraph` 库（状态图+条件边+状态持久化+自动生成流程图）实现了一个三分支判决的评审流水线，并且专门讲清楚了"这个场景为什么真的需要框架、而不是像 Demo06 一样手写"这条判断逻辑。

下一章，我们把"不需要框架"这个观点付诸实践——从零写一个不到 300 行、但能覆盖 ReAct 循环+工

具注册+基础记忆的最小 Agent 框架。

## 章节自测（5道单选，答案见文末）

Q1. 产品经理评测框架该关心的核心问题是什么？ A. GitHub Star数 B. 能否长期维护、出问题能否快速定位、迁移成本高低 C. 论文引用量 D. 是哪个大公司出的

Q2. LangGraph 的核心抽象是什么？ A. 角色对话 B. 状态图（节点+边） C. 消息队列 D. 数据库表

Q3. 给自己留后路避免框架深度绑定的做法是什么？ A. 完全不用框架 B. 把业务逻辑写成独立于框架的纯函数，框架只负责编排调用顺序 C. 只用一个框架不切换 D. 把所有逻辑写进框架回调

Q4. AutoGen 的核心抽象是什么？ A. 状态图 B. 可对话的角色（Conversable Agent） C. 分布式运行时 D. 规则引擎

Q5. 文中给出的反常识建议是什么？ A. 一定要用框架 B. 很多场景根本不需要框架，代码量小能完全掌控反而调试更快 C. 框架越复杂越好 D. 框架选错了没法挽回

► [点击查看参考答案](#)